CodeV: Empowering LLMs with HDL Generation through Multi-Level Summarization

Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yinan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, Yansong Pan, Zhenxing Zhang, Rui Zhang, *Member, IEEE*, Xishan Zhang, Zidong Du, *Member, IEEE*, Qi Guo, *Member, IEEE*, Xing Hu, *Member, IEEE*

arXiv:2407.10424v5 [cs.PL] 11 May 2025

***Abstract*—The design flow of processors, particularly in hardware description languages (HDL) like Verilog and Chisel, is complex and costly. While recent advances in large language models (LLMs) have significantly improved coding tasks in software languages such as Python, their application in HDL generation remains limited due to the scarcity of high-quality HDL data. Traditional methods of adapting LLMs for hardware design rely on synthetic HDL datasets, which often suffer from low quality because even advanced LLMs like GPT perform poorly in the HDL domain. Moreover, these methods focus solely on chat tasks and the Verilog language, limiting their application scenarios.**

In this paper, we observe that: (1) HDL code collected from the real world is of higher quality than code generated by LLMs. (2) LLMs like GPT-3.5 excel in summarizing HDL code rather than generating it. (3) An explicit language tag can help LLMs better adapt to the target language when there is insufficient data. Based on these observations, we propose an efficient LLM fine-tuning pipeline for HDL generation that integrates a multi-level summarization data synthesis process with a novel Chat-FIM-Tag supervised fine-tuning method. The pipeline enhances the generation of HDL code from natural language descriptions and enables the handling of various tasks such as chat and infilling incomplete code. Utilizing this pipeline, we introduce CodeV, a series of HDL generation LLMs. Among them, CodeV-All not only possesses a more diverse range of language abilities, *i.e.* Verilog and Chisel, and a broader scope of tasks, *i.e.* Chat and fill-in-middle (FIM), but it also achieves performance on VerilogEval that is comparable to or even surpasses that of CodeV-Verilog fine-tuned on Verilog only, making them the first series of open-source LLMs designed for multi-scenario HDL generation.

***Index Terms*—Hardware design languages (HDLs) generation, Large language models (LLMs), Processor design automation.**

Yang Zhao, Chongxiao Li, Pengwei Jin, Ziyuan Nan, Mingju Gao, Yansong Pan are with the State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, the University of Chinese Academy of Sciences, Beijing, China, and also with Cambricon Technologies.

Di Huang, Yinan Xu, Rui Zhang, and Qi Guo are with the State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China.

Muxin Song is with the School of Information Science and Technology, ShanghaiTech University, Shanghai, China.

Tianyun Ma, Lei Qi, Zhenxing Zhang are with the University of Science and Technology of China, Hefei, China, the State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China.

Zidong Du and Xing Hu are with the State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, and Shanghai Innovation Center for Processor Technologies, Shanghai, China.

Xishan Zhang is with the State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, and also with Cambricon Technologies.

I. Introduction

THE design flow of processors is expensive and complex, especially the coding of the hardware description language (HDL) like Verilog and Chisel [1]–[3]. Recent advances in large language models (LLMs) have not only delivered impressive performance in coding tasks for software languages such as Python [4]–[10] but have also unveiled promising avenues for the direct generation of HDL from natural language descriptions. However, the good performance of LLMs on software coding tasks is attributed to the large amount of high-quality description-code data available in the software field, whereas in the field of hardware design, high-quality description-code HDL data is very scarce. For example, in the largest open-source code dataset, The Stack v2 [11], the Scala data accounts for only 10.1M (with Chisel constituting only a small part of the Scala language), the Verilog data is merely 1.91M, whereas the Python data amounts to 80.6M – a difference of several tens of times.

Although initial efforts have explored adapting LLMs for hardware design by fine-tuning them on synthesized Verilog data [12]–[17], these approaches have not yet effectively addressed this problem. This shortfall primarily stems from current data synthesis methods, which heavily depend on advanced LLMs—such as GPT—to generate both descriptions and corresponding code. However, because even these advanced LLMs perform poorly in the HDL domain, the quality of the synthesized data remains low. For instance, while GPT-4 achieves an 88.4% pass@1 score on the Python benchmark HumanEval [18], [19], it only reaches 43.5% on the VerilogEval-Human benchmark [20]. Moreover, existing methods focus solely on chat tasks and the Verilog language—that is, they generate Verilog code from natural language descriptions—and are not equipped to handle additional scenarios, such as infilling incomplete code, or other languages, such as Chisel.

Regarding these challenges, we have observed the following: **(1) HDL code collected from the real world is of higher quality than that generated by LLMs, and LLMs excel in summarizing HDL code rather than generating it.** HDL code collected from the real world has fewer syntax errors, clearer semantics, and better conforms to the code distribution in practical applications compared to HDL code generated by LLMs. Figure 2a-c illustrates an example observed from the GPT-generated HDL dataset from RTLCoder [12]. Figure 2a shows a description obtained by combining two circuit descrip-

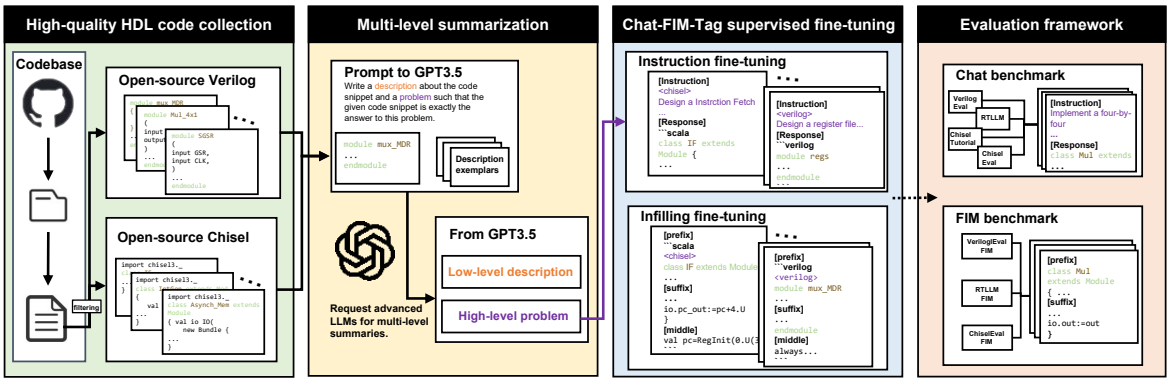

Fig. 1. **The CodeV framework overview.** We first collect and filter high-quality HDL modules from open-source codebases. The modules are then sent to GPT-3.5 to request multi-level summaries. Pairing high-level descriptions with corresponding modules, the high-quality dataset is utilized to fine-tune base LLMs, yielding CodeV models.

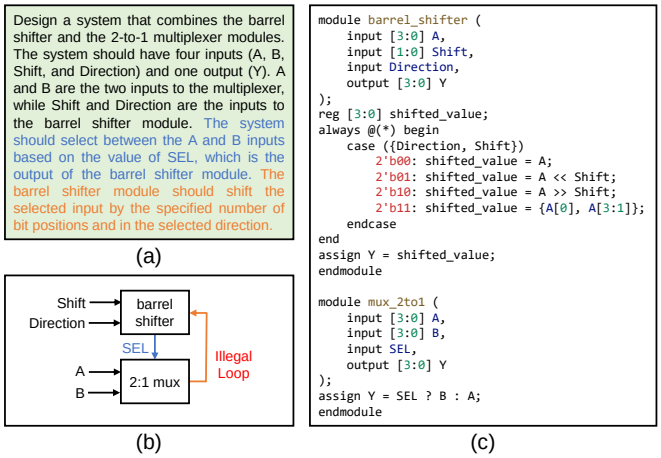

Fig. 2. GPT-generated Verilog dataset contains unrealistic code. (a) A synthetic description obtained from RTLCoder [12], which is unrealistic. (b) The circuit's block diagram according to the description, which contains illegal loops in the circuit. (c) The corresponding incorrect Verilog code in the dataset.

tions. We draw a block diagram based on this description, and as shown in Figure 2b, this description leads to a loop in a combinational circuit, which is illegal. Figure 2c depicts the circuit generated by GPT-3.5 based on this incorrect description, which fails to integrate the shifter and multiplexer, and the internal implementation of the shifter module is also incorrect. **(2) LLMs excel in summarizing HDL code rather than generating it.** As shown in Table I, where we manually assess whether the descriptions produced by GPT-3.5 accurately correspond to the Verilog code, although GPT-3.5's poor performance in generating Verilog (33.5% pass@1), it indeed offers the ability to describe the functionality of given Verilog code (64.7% pass@1). **(3) An explicit language tag can help LLMs better adapt to the target language when there is insufficient data.** For example, when generating both Verilog and Chisel with even scarcer data, using the language tags "`<Verilog>`" and "`<Chisel>`" to differentiate between them can simultaneously enhance the model's performance in both languages. A detailed supporting experiment is shown in Table XI.

TABLE I
SUMMARIZING VERILOG V.S. GENERATING VERILOG.

Task	Pass@1 (%)
Verilog Summarization	64.7
Verilog Generation	33.5

Based on these observations, we propose an efficient LLM fine-tuning pipeline that targets HDL generation. As shown in Figure 1, this pipeline includes a data synthesis process and a training process. For the data synthesis process, we employ **multi-level summarization** for generating the high-quality dataset from raw code. Instead of synthesizing descriptions first and then getting the corresponding code from an advanced LLM, we prompt the LLM with HDL code and let the LLM generate the corresponding natural language descriptions. For the fine-tuning process, to more closely align with real-world design workflows, we use the Chat-FIM (*i.e.* fill-in-middle) hybrid fine-tuning method with a language tag that enables LLMs to effectively handle different languages even when limited data is available.

Building upon this pipeline, we introduce CodeV, a series of instruction-tuned HDL generation LLMs. Specifically, in the data synthesis process, we first crawl over 1.5M Verilog modules and 24K Chisel modules from GitHub, filter them, and obtain a deduplicated, self-contained, and syntax-correct set containing 165K high-quality Verilog modules and 18.7K Chisel modules. Based on these HDL modules, we use GPT-3.5 to generate fine-grained functional descriptions that more accurately capture the functionality of HDL, although there remains a gap with real-world use cases. We then let GPT-3.5 summarize high-level function specifications, which are closer to real-world scenarios, thereby obtaining a high-quality dataset of description-code pairs. In the fine-tuning process, we employ the Chat-FIM-Tag supervised fine-tuning method, which enables CodeV to handle both Chat (generating code based on natural language descriptions) and FIM (infilling incomplete code) tasks and two different HDL, Verilog and Chisel.

Additionally, to evaluate the generation and FIM ability

of LLMs on Chisel, we propose two Chisel benchmarks, ChiselTutorial and ChiselEval, and extend existing Verilog benchmarks to an FIM format. Among them, ChiselTutorial is a collection of introductory Chisel problems, and ChiselEval is the Chisel version of the VerilogEval translation. Experimental results show that CodeV-All not only possesses a more diverse range of language abilities and a broader scope of tasks, but it also achieves performance on VerilogEval that is comparable to or even surpasses that of CodeV-Verilog, making them the first series of open-source LLMs designed for multi-scenario HDL generation. Specifically, CodeV-All-QC achieves state-of-the-art (SOTA) results within open-source LLMs on both VerilogEval [20] and RTLLM [21], and surpasses the ever-reported SOTA LLM CraftRTL [15] by 4.1% pass@1 and the commercial LLM GPT-3.5 by 35.2% pass@1 on VerilogEval-Machine.

The major contributions of this paper are as follows:

- We propose an efficient LLM fine-tuning pipeline that targets HDL generation, including the multi-level summarization data synthesis process and Chat-FIM-Tag supervised fine-tuning process. This pipeline enables the generation of HDL for general scenarios.
- Building on this method, we introduce a series of state-of-the-art (SOTA), open-source Verilog generation LLMs, named CodeV-Verilog. Notably, CodeV-Verilog-QC achieves an 80.1% pass@1 on the VerilogEval-Machine benchmark and 59.2% on the VerilogEval-Human benchmark—surpassing GPT-3.5, GPT-4. It also attains a 96.6% syntax pass rate and a 51.7% function pass rate on the RTLLM benchmark, outperforming RTLCoder, the previous SOTA open-source LLM.
- We propose two Chisel benchmarks, ChiselTutorial and ChiselEval, and extend them along with existing Verilog benchmarks to an FIM format. These benchmarks provide a more comprehensive evaluation process for LLM-based HDL generation.
- To show the generalizability of our pipeline, we introduce CodeV-All, the first open-source LLM series designed for multi-scenario HDL generation. It demonstrates remarkable performance across different HDL languages (Verilog and Chisel) and multiple tasks (Chat and FIM), showcasing the generalization capability of our method.
- We plan to open-source our dataset, which contains 185K high-quality description-code pairs (165.3K for Verilog and 18.7K for Chisel), to foster advancement and collaboration within the LLM, electronic design automation (EDA), and programming language communities.

II. Related Work

A. LLMs for HDL Generation

Thakur et al. [22] collected approximately 400MB of Verilog files from GitHub and textbooks to fine-tune CodeGen-16B [23]. However, the dataset was not refined, resulting in a blend of text and code, with some Verilog code potentially containing syntax errors. ChipNemo [24] used LLaMA2 [25] as its base model, performing domain-adaptive pre-training on public datasets and NVIDIA's internal chip design files (including HDL code, EDA scripts, and configuration files, etc.), and fine-tuning on public instruction datasets and an expert-crafted domain dataset with 1430 examples. However, likely due to the dataset not being refined for Verilog generation tasks, the 70B version model of ChipNemo even underperforms its base model on the VerilogEval-Human benchmark. RTLCoder [12] uses GPT to synthesize instruction-code pairs, starting by generating instructions based on a keywords pool and a source code pool, then mutating these instructions to increase their number and diversity, and finally having GPT generate Verilog code based on these synthesized instructions to form a fine-tuning dataset. This approach may lead to unrealistic instructions, and the code quality is limited by GPT's generation ability. BetterV [13] collects Verilog code from GitHub and expands its dataset by translating Verilog code into C to create C-Verilog pairs, leveraging the base model's knowledge in C to enhance Verilog generation capabilities. AutoVCoder [14] collects data from up to 20,000 GitHub repositories based on its proposed automated dataset generation method, and generates corresponding question-code pairs through LLM, using different keywords to increase code diversity. OriGen [17] collected open-source RTL code samples and generated detailed descriptions of the corresponding code using Claude3-Haiku. It then regenerated RTL code based on the detailed descriptions to build a code enhancement dataset, and proposed an error correction dataset by modifying parts of the main content of the code. CraftRTL [15] identified the difficulties of non-text representations in synthetic data generation and the variability in performance during benchmark training. It proposed a targeted fine-tuning dataset for LLMs, which faced challenges in solving Karnaugh maps, state transition diagrams, and waveform issues in Verilog code problems, and significantly alleviated the model's performance on related issues. HAVEN [16] addresses issues such as symbolic illusion, knowledge illusion, and logical illusion. Based on the proposed illusion classification method, it constructs knowledge-enhanced and logic-enhanced datasets, and proposes a symbolic understanding chain method that integrates chain-of-thought (CoT). In parallel with our work, MG-Verilog [26] uses the summarization technique to create a multi-grained description-Verilog dataset. However, our focus is on different languages (*i.e.* Verilog and Chisel) and scenarios (*i.e.* Chat and FIM). While MG-Verilog aims to generate Verilog code from varying levels of descriptions, leading them to modify the original VerilogEval benchmark to fit their needs, our goal is to enhance the overall performance of HDL generation models utilizing the mismatch between generation and summarization ability and demonstrate their potential in practical module design.

In the field of hardware circuit design, traditional low-level hardware description code design related to HDL such as Verilog is difficult to meet the requirements for code readability, modularity, and scalability. The Chisel [27], which adopts an object-oriented approach to circuit design, based on the Scala language has emerged. ChatChisel [28] implemented the design of an RV32I RISC-V CPU with the help of GPT-3.5-Turbo, achieving a 31.86% performance improve-

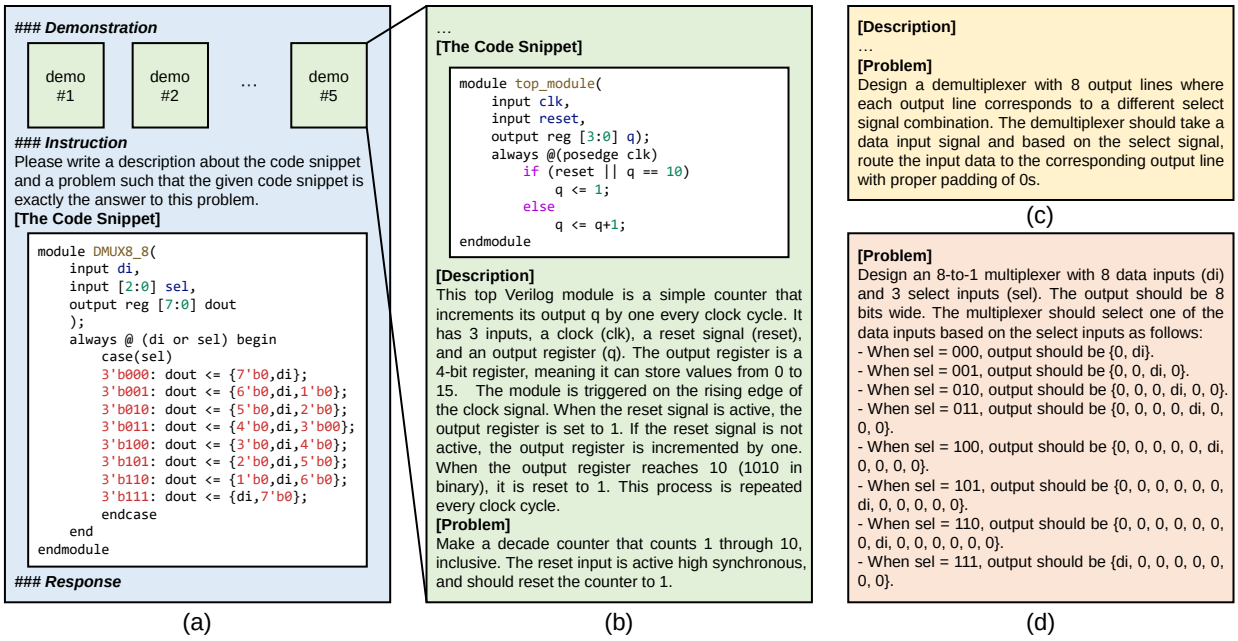

Fig. 3. An actual example of the prompt for multi-level summarization. (a) The prompt provided to GPT-3.5. (b) An example of the demonstrations, with code, low-level descriptions, and high-level summaries. (c) Summaries responded from GPT-3.5 with and (d) without multi-level summarization.

ment over Verilog-based hardware design levels. However, the data scarcity challenge of Chisel is even more severe than that of Verilog, which leads to poor performance on fine-tuning. Moreover, there is a lack of relevant benchmarks to measure the hardware circuit generation capabilities of LLMs on Chisel.

B. Frameworks on HDL Generation

Instead of training specific models, some work focuses on designing generation frameworks for HDL tasks to improve general-purpose LLMs' performance. Chip-Chat [29] and ChipGPT [30] use multi-round EDA and human feedback to assist LLMs for Verilog generation. Lu et al. [21] introduced a self-planning method to improve generation accuracy, where LLMs plan before generating Verilog. OriGen [17] uses feedback loop to generate Verilog and incorporates a teacher-student learning framework to correct the dataset in order to help the model better understand error messages. AutoVCoder [14] introduces retrieval-augmented generation (RAG) technology into Verilog design problems and designs a framework containing an example retriever and a knowledge retriever. RTLFixer [31] combines RAG and ReAct prompts to automatically fix syntax errors in Verilog code using large language models. Andre Nakkab et al. [32] proposed a hierarchical prompting technique that uses a layered approach to automatically generate complex hardware modules and completed a processor entirely designed by LLM. Verilog-Coder [33] proposed a system composed of multiple agents that can autonomously write Verilog and use Verilog tools to fix syntax and functional errors. AIvril [34] designed a framework that combines a Code Agent and a Review Agent to write Verilog code and testbench, and to perform automatic self-correction and functional verification. RTL Agent [35] explored high-quality Verilog generation tasks by designing a Reflexion workflow generated by Verilog. These methods are orthogonal to our work, which forms the foundation for them, as they all require a code model capable of generating HDL.

In contrast to these methods, CodeV employs a novel dataset construction approach with a validated insight into the difficulty of code generation and code summarization, utilizing multi-level summarization to generate high-quality instruction tuning datasets from high-quality Verilog code, achieving SOTA results.

III. Methods

A. High-quality HDL Code Collection

a) Verilog code collection: High-quality real-world Verilog code is the foundation of our approach. We first crawl Verilog and SystemVerilog files from GitHub, ensuring each file contains a complete module, i.e. contains at least one "`module`" and "`endmodule`" pair. Files with external references, *i.e.* code containing "`include`" or "`import`" keywords, are excluded. This is because reconstructing these reference relationships from the crawled files is challenging, and we do not want LLMs to generate these non-self-contained Verilog codes after training. Then we deal with the comments interspersed throughout the files. These comments can be categorized into two types: those related to Verilog implementation, such as signal descriptions, module explanations, test cases, etc.; and those unrelated to implementation, such as licenses, company and author information, logs, etc. The former is essential for LLMs to understand the code and should be kept, while the latter should be avoided. Therefore, we use regular expressions to delete these implementation-unrelated comments. Next, files exceeding 4096 characters are deleted,

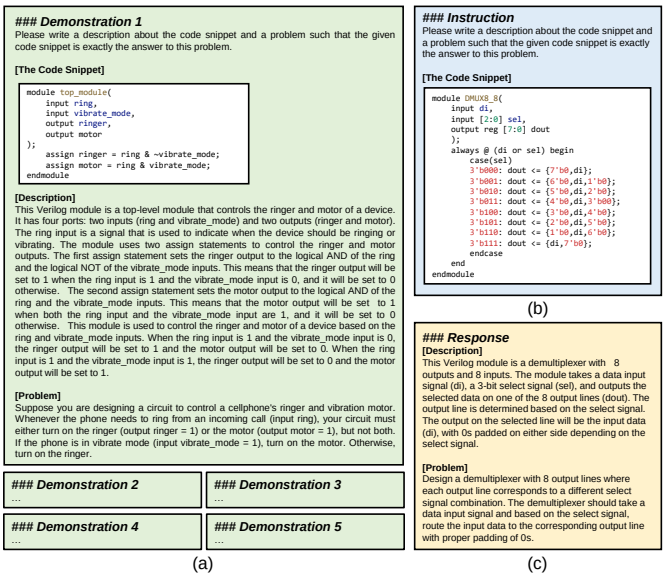

Fig. 4. A detailed example showing all prompts used for multi-level summarization with Verilog dataset. (a) Demonstrations given to GPT-3.5. (b) A Verilog data snippet in CodeV generating corresponding Instructions, which can be adjusted by modifying the Code Snippet. (c) The response from GPT-3.5, consisting of the Description and Problem sections.

as longer files would be truncated during training, resulting in incomplete file content and degrading the dataset quality.

b) Chisel code collection: In order to enable our model to have multi-lingual capabilities, we create a Chisel dataset. Similar to the method used for creating the Verilog dataset, we obtain 1,651 repository projects related to Chisel currently on GitHub through the GitHub API. We extract Chisel code from these projects. We retain files with the ".scala" extension that import the Chisel package in their content. These characteristics distinguish hardware design code written in the Chisel language from others.

c) Data deduplication: This process aims to filter out similar data in the training dataset. Both Verilog and Chisel employ the same deduplication method. Following [22], we also use MinHash and Jaccard similarity to filter out duplicate code. Each Verilog file is hashed into a 128-dimensional vector using the MinHash API provided by datasketch [36]. We go through the files in order, and if the Jaccard similarity between a Verilog file and any preceding file exceeds a certain threshold, the file is considered a duplicate and filtered out. In our processing, this threshold is set to 0.8.

d) Quality enhancing: To further enhance the quality of the dataset, we compile each file with Icarus Verilog [37] for automatic syntax checking. Files that fail to compile are removed.

B. Multi-level Code Summarization

As mentioned before, manual annotation is prohibitively time-consuming and costly. Hence, we use GPT-3.5 to generate high-level summaries for each Verilog and Chisel module as its design specification. As analyzed in VerilogEval [20], when required for summarizing, LLMs often produce verbose descriptions, preferring line-by-line explanations over high-level summaries. To address this issue, we introduce a multi-level summarization method, employing few-shot learning [38] to guide GPT-3.5 in first producing detailed descriptions and then abstracting high-level summaries.

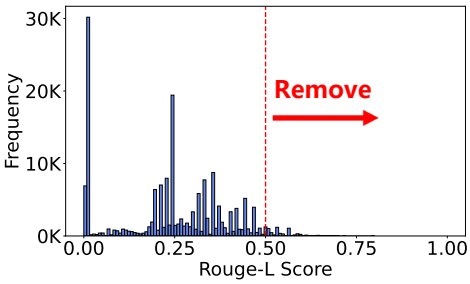

Fig. 5. The similarity distribution of our dataset against benchmark data. Data with Rouge-L scores greater than 0.5 are removed for decontamination.

Figure 3 presents a Verilog example from our actual process, involving the generation of requirement descriptions for a demultiplexer module collected from GitHub. Figure 3a illustrates our prompt, which includes n examples with code snippets, detailed descriptions, and problem summaries, followed by the demultiplexer code, and a request for GPT to imitate the examples by first providing a detailed description and then a problem summary. Figure 3b illustrates an example used for GPT's few-shot learning. Figure 3c shows GPT's response using multi-level summarization, while Figure 3d shows the response without multi-level summarization by deleting the detailed description in the demonstrations. We observe that the description in Figure 3c is more concise and aligns with actual Verilog design specifications. In contrast, the description in Figure 3d is verbose and even inaccurately counts the number of zeros in output signals, leading to a contradictory explanation.

The prompt details are shown in Figure 4 where we add more details about one of the 5 demonstrations. The 5 demonstrations are "ringer" (fully displayed), "dff16e", "count1to10", "lfsr5", and "gatesv100".

C. Data Decontamination

To ensure a fair evaluation, we conduct decontamination for both Verilog and Chisel training data to remove sequences similar to benchmarks (details about our Chisel benchmark is shown in Section IV). Specifically, we use Rouge-L [39], a widely used similarity metric, to measure the similarity between data:

$$ROUGE-L(C^i_{train}) = \max_j\left(\frac{(1+\beta^2)LCS(C^i_{train}, C^j_{test})}{len(C^i_{train}) + \beta^2 len(C^j_{test})}\right), \tag{1}$$

where C^i_{train} denotes the i-th Verilog code from the training data, C^j_{test} denotes the j-th Verilog code in the test data, and $LCS(C^i_{train}, C^j_{test})$ denotes the length of a longest common subsequence of C^i_{train} and C^j_{test}. $ROUGE-L(C^i_{train}) \in [0, 1]$, with higher values indicating greater similarity. Following RTLCoder [12], we set $\beta = 1.0$ and remove data from the

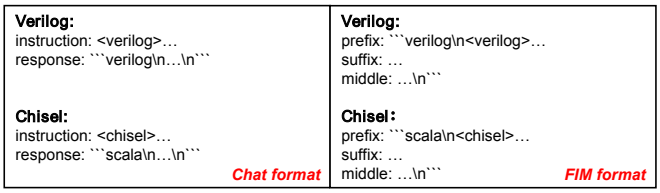

Fig. 6. The training data format. Here, "`<verilog>`" and "`<chisel>`" are used to identify the language of the task, while "`'''verilog\n...\n'''`" and "`'''scala\n...\n'''`" are used to mark the scope of code segments.

training set that have a Rouge-L score greater than 0.5 with any of the benchmarks, thereby preventing test data leakage. The Rouge-L plot is shown in Figure 5. Through the above-mentioned effort, we obtain high-quality real-world Verilog code. Next, we will introduce the fine-tuning method.

D. Chat-FIM-Tag supervised fine-tuning

We introduce our Chat-FIM-Tag supervised fine-tuning method to alleviate the challenges of multi-scenario chat and infilling tasks, and that different languages may affect each other's performance when the training data is scarce.

Due to the inherent contextual dependencies in programming languages, relying solely on the next-token prediction of LLMs is insufficient to achieve satisfactory FIM performance. To enhance the FIM capability of the model, we conduct infilling fine-tuning following [40]. Specifically, we randomly partition each training document into prefix, middle, and suffix sections, then concatenate them using special FIM tokens. Both line-level and character-level spans are employed for the partition, maintaining a 2:1 ratio between them. There are two distinct infilling modes for FIM method: PSM (Prefix-Suffix-Middle) and SPM (Suffix-Prefix-Middle). The PSM mode concatenates sections in the order of prefix, suffix, and middle, which is particularly suited for scenarios where both the prefix and suffix are non-empty on either side of the middle section. Conversely, the SPM mode follows the sequence of suffix, prefix, and middle for concatenation. According to [41], [42], we employ the PSM (Prefix-Suffix-Middle) mode for infilling fine-tuning. Finally, we have processed the training document into the following tokenized version:

`<PRE>{prefix}<SUF>{suffix}<MID>{middle}<EOT>`

where "`<PRE>`", "`<MID>`", "`<SUF>`", and "`<EOT>`" need to be modified according to the model's tokenization. It is noteworthy that we preserve the loss for all the prefix, middle, and suffix sections in this study.

When the data for a single language is limited, including multiple languages can affect the performance of that language. To address this challenge, we have empirically found that emphasizing the distinctions between languages during training helps improve the performance of each language. Specifically, we incorporate language-specific tags, that is "`<Verilog>`" and "`<Chisel>`", during training.

Based on these, we construct two training datasets, as presented in Table II. To enable models to better distinguish between different language tasks and identify the start and end positions of code, we add tags to the code in the 184k-Verilog-Chisel dataset before the tokenization process. The format can refer to Figure 6.

TABLE II
COMPOSITION OF THE TRAINING DATASETS.

Dataset	Verilog (k)	Chisel (k)	FIM rate (%)
165k-Verilog	165.3	0	0
184k-Verilog-Chisel	165.3	18.7	33.3

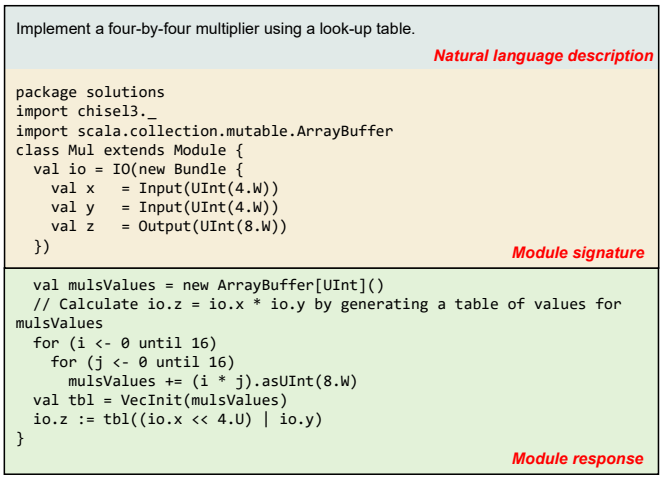

Fig. 7. An example from ChiselTutorial.

Finally, we fine-tune base models on our datasets. Given a natural language description, the LLM with parameter θ generates code repeatedly drawing from $p_\theta(x_t|x_{<t})$ and using x_t as part of the input for the subsequent prediction. Following prior work, the LLM is designed to reduce the negative log-likelihood across the dataset D $\{x^1, ..., x^{|D|}\}$ [43]:

$$\mathcal{L} = -\frac{1}{|D|}\sum_{i=1}^{|D|}\frac{1}{n}\sum_{t=1}^{n}\log p_\theta(x_t^i|x_{<t}^i). \tag{2}$$

IV. EVALUATION FRAMEWORK

In this section, we introduce our benchmarks based on the proposed Chat and FIM testing tasks. In the first part, we modify the open-source Chisel tutorial to design the ChiselTutorial benchmark for Chisel evaluation. Additionally, we adapt the benchmark problems from VerilogEval, considering the diverse hardware programming challenges it covers, to create ChiselEval for Chisel evaluation. In the second part, we reformat three Verilog and Chisel benchmarks, VerilogEval, RTLLM, and ChatEval, to establish FIM benchmarks for FIM evaluation, namely VerilogEval-FIM, RTLLM-FIM, and ChatEval-FIM. Details are introduced as follows.

A. Chat Benchmark

a) ChiselTutorial Benchmark: We find that the current LLMs still have a lot of syntax errors in Chisel programming capabilities. Therefore, it is necessary to conduct the most

basic Chisel introductory tests. We modify ChiselTutorial[1], an official programming exercise of Chisel, to form our ChiselTutorial benchmark which evaluates the basic Chisel capabilities of LLMs. ChiselTutorial is an official example code presented for beginners learning Chisel, aiming to demonstrate the latest Chisel programming syntax to Chisel novices. This example includes some relatively simple Chisel programming tasks, covering basic hardware design problems from multiplexers and arithmetic operations to counters. Specifically, we extracted 17 programming problems from the ChiselTutorial as evaluation problems in our benchmark. We used the original code as the answers and then extracted and modified the comments that introduce the problems in the original code as the natural language descriptions. After each problem description, we added the module header definitions from the original code file to ensure that the generated Chisel code can be accurately called by the testbench. For evaluation, we use the official testbench from ChiselTutorial and run the code using the Scala build tool set. We provide reference correct code and include inputs that can be processed by LLMs along with the corresponding module headers. An example problem of ChiselTutorial is shown in Figure 7.

b) ChiselEval Benchmark: To further evaluate the Chisel programming ability of LLMs, we design the ChiselEval benchmark. Referring to the problem structure in VerilogEval, we create ChiselEval by rewriting the answers to all 156 problems in VerilogEval-Human to Chisel. Similar to VerilogEval, ChiselEval also extensively covers fundamental problems in hardware circuit design, such as basic signal processing, multiplexer design, and so on, ensuring that our designed benchmarks align more closely with actual design problems in Chisel. We modify problems in VerilogEval to make them suitable for Chisel generation. When adapting the natural language specifications in the problems, we ensure that the circuit functionality and logical implementation remain consistent with the original VerilogEval benchmark. We retain and adapt problems involving graphs or tables in VerilogEval, such as those with Karnaugh maps or truth tables, to evaluate the model's ability to design circuits based on these non-textual contents. For each design, our benchmark also provides the corresponding verification environment. When all test cases match the expected output in the top module, it indicates that the LLM has correctly generated the code.

B. FIM Benchmark

We introduce three FIM benchmarks: VerilogEval-FIM, RTLLM-FIM, and ChiselEval-FIM, which are constructed by adapting VerilogEval [20], RTLLM [21], and ChiselEval. These benchmarks are specifically designed to evaluate the FIM capabilities of models in the Verilog and Chisel. Following [40], [44], we implement single-line, multi-line, and random-span infilling for each FIM benchmark.

In the case of single-line infilling, we mask one non-empty line of the canonical code implementation to generate the FIM task. Multi-line infilling adopts a similar approach, with the distinction that the masked region may contain multiple non-empty lines. With random-span infilling, we randomly mask a non-empty middle span at the character level. From each canonical code implementation, we randomly select one instance from each of the three generated task types to serve as the final FIM (Fill-in-Middle) task, ensuring each implementation yields exactly one FIM task for each infilling type while maintaining uniformity across all types.

[1]https://github.com/ucb-bar/chisel-tutorial

```
<PRE>module pe(
    input clk,
    input rst,
    input [31:0] a,
    input [31:0] b,

    output [31:0] c
);

<SUF>  end

end

endmodule
<MID>
```

FIM prompt

```
reg [31:0] c;

always@(posedge clk or posedge rst)
begin

  if(rst)
  begin
    c <= 0;
  end
  else
  begin
    c <= a + b;
```

Response

Fig. 8. An example of a VerilogEval-FIM testing framework.

For the three benchmarks, we preserve the module headers from their canonical code implementations to ensure testing reliability, avoiding potential issues caused by variations in generated module header names. For each circuit across the benchmarks, we establish tasks corresponding to all three infilling types. Consequently, the VerilogEval-FIM Benchmark consists of two parts: VerilogEval-FIM-Machine and VerilogEval-FIM-HUMAN, containing 429 and 468 tasks respectively. RTLLM-FIM comprises a total of 87 tasks, and the ChiselEval-FIM Benchmark contains 468 tasks. The distribution among the three infilling types is balanced, with equal numbers for each type.

Taking the VerilogEval-FIM Benchmark as an example, the test format is illustrated in Figure 8. In this format, the tokens "`<PRE>`", "`<MID>`", and "`<SUF>`" need to be modified according to the model's specific FIM tokens. Through these three FIM benchmarks, the model's code-infilling capabilities in both Verilog and Chisel hardware programming languages can be effectively evaluated.

V. Experiments

We conduct comprehensive experiments to investigate that:

- How does CodeV-Verilog, LLMs fine-tuned only with Verilog data, perform on Verilog across multiple benchmarks and settings (Section V-B)?
- How does CodeV-All, fine-tuned with Verilog and Chisel data, perform on multi-lingual and multi-scenario tasks (Section V-C)?
- The effectiveness of multi-level summarization and Chat-FIM-Tag supervised fine-tuning (Section V-D).
- Further analysis on the impact of dataset size, and sampling temperature (Section V-E).

A. Experimental Setup

1) Supervised Fine-tuning: We use CodeLlama-7b-Instruct [45], Deepseek-Coder-6.7b-Instruct [41], CodeQwen1.5-7B-Chat [46] and Qwen2.5-Coder-7B [42] as our base models.

TABLE III
COMPARISON OF OUR CODEV MODELS AGAINST VARIOUS BASELINE MODELS. ACCURACY DATA ARE CITED FROM THEIR ORIGINAL PAPER [12]–[17], [24]. * ARE THE RESULT UPDATED BY US WITH THE NEW VERSION.

Type	Model	Open source	VerilogEval-Machine (%)			VerilogEval-Human (%)			RTLLM v1.1 (%)	
			pass@1	pass@5	pass@10	pass@1	pass@5	pass@10	Syntax	Func.
Foundation models	GPT-3.5	×	46.7	69.1	74.1	26.7	45.8	51.7	89.7	37.9
	GPT-3.5-turbo-0125*	×	60.9	75.0	79.9	33.5	45.9	50.0	79.3	51.7
	GPT-4	×	60.0	70.6	73.5	43.5	55.8	58.9	**100.0**	65.5
	StarCoder	✓	46.8	54.5	59.6	18.1	26.1	30.4	93.1	27.6
	CodeLlama	✓	43.1	47.1	47.7	18.2	22.7	24.3	86.2	31.0
	DeepSeek-Coder	✓	52.2	55.4	56.8	30.2	33.9	34.9	93.1	44.8
	CodeQwen	✓	46.5	54.9	56.4	22.5	26.1	28.0	86.2	41.4
	Qwen2.5-Coder	✓	66.2	79.2	83.9	34.6	45.6	51.0	89.6	41.4
IT baselines	ChipNeMo	×	43.4	-	-	22.4	-	-	-	-
	Thakur et al.	✓	44.0	52.6	59.2	30.3	43.9	49.6	86.2	24.1
	RTLCoder-Mistral	✓	62.5	72.2	76.6	36.7	45.5	49.2	96.6	48.3
	RTLCoder-DS	✓	61.2	76.5	81.8	41.6	50.1	53.4	93.1	48.3
	BetterV-CL	×	64.2	75.4	79.1	40.9	50.0	53.3	-	-
	BetterV-DS	×	67.8	79.1	84.0	45.9	53.3	57.6	-	-
	BetterV-CQ	×	68.1	79.4	84.5	46.1	53.7	58.2	-	-
	CraftRTL-CL	×	78.1	85.5	87.8	63.1	67.8	69.7	93.9	52.9
	CraftRTL-DS	×	77.8	85.5	88.1	**65.4**	**70.0**	**72.1**	84.3	58.8
CodeV-Verilog	CodeV-Verilog-CL(Ours)	✓	78.1	86.0	88.5	45.2	59.5	63.8	93.1	**62.1**
	CodeV-Verilog-DS(Ours)	✓	77.9	88.6	90.7	52.7	62.5	67.3	89.7	55.2
	CodeV-Verilog-CQ(Ours)	✓	77.6	88.2	90.7	53.2	65.1	68.5	93.1	55.2
	CodeV-Verilog-QC(Ours)	✓	80.1	87.9	90.5	59.2	65.8	69.1	96.6	51.7
CodeV-All	CodeV-All-CL(Ours)	✓	78.5	85.6	87.6	46.6	58.8	62.5	96.6	55.2
	CodeV-All-DS(Ours)	✓	79.8	86.0	86.7	53.0	63.3	67.2	96.6	51.7
	CodeV-All-CQ(Ours)	✓	79.9	88.3	91.1	54.1	65.1	68.6	93.1	58.6
	CodeV-All-QC(Ours)	✓	**81.9**	**89.9**	**92.0**	56.6	67.9	71.4	96.6	55.2

We fine-tune these models with full parameters for 3 epochs using PyTorch's Distributed Data Parallel (DDP) on 4 NVIDIA A100-80GB SMX GPUs. We employ the Adafactor optimizer [47] with a linear learning rate scheduler, setting the initial learning rate at $5e-5$ with 15 warm-up steps. We set global batch size 256, with max sequence length 2048 which follows the setting of previous work [12], [48].

2) Benchmarks and Metrics: We conduct comprehensive evaluations on the VerilogEval [20], RTLLM [21], ChiselTutorial, ChiselEval, VerilogEval-FIM, RTLLM-FIM and ChiselEval-FIM benchmarks.

VerilogEval includes two parts: VerilogEval-Machine and VerilogEval-Human, with 143 GPT-generated and 156 hand-crafted Verilog generation problems, respectively. We follow the VerilogEval paper, using the pass@k metric to measure the Verilog generation accuracy. The pass@k metric estimates the proportion of problems that can be solved at least once in k attempts:

$$\mathbf{pass@}k := \mathop{\mathbb{E}}_{\text{problems}} \left[\frac{1 - \binom{n-c}{k}}{\binom{n}{k}} \right] \tag{3}$$

where $n \geq k$ represents the total number of trials for each problem, and c represents the number of trials that pass the functional check. We set $n = 20$ following previous work.

RTLLM contains 29 Verilog generation problems. Following prior work [12], we test models on RTLLMv1.1 which is error-fixed. Also, following the original settings, we generate Verilog code for each problem 5 times and report the success rate (proportion of problems that at least one trial passes the syntax check and function check, respectively). In addition, we also provide a more comprehensive evaluation of the RTLLM benchmark in Section V-E3, setting $n = 20$ for each generation task which is similar to VerilogEval.

Following RTLCoder [12], for all tests, we set the generation temperature of each model to $\{0.2, 0.5, 0.8\}$, and report the best performance across different temperature settings, but we also present results on three temperatures separately in Section V-E2.

Details on ChiselTutorial, ChiselEval, VerilogEval-FIM, RTLLM-FIM, and ChiselEval-FIM can refer to Section IV-A and IV-B. We evaluate model performance using the pass@k metric with $n = 20$, except for RTLLM-FIM, which follows the same evaluation metric as RTLLM (perform code infilling for each problem 5 times and report the success rate). We fix the temperature at 0.2 to compare the results of different models.

In all the results, We use **bold** to denote the best model overall and underline to indicate the second-best results overall.

3) Baseline Methods: In our experiment, we compare our results with baseline methods on ∼7B LLMs, including: (1) Foundation models not tailored for Verilog generation tasks, including closed-source commercial LLMs such as GPT-3.5 and GPT-4, and four open-source models specialized in code generation tasks: StarCoder [49], CodeLlama-7b-Instruct [45], DeepSeek-Coder-6.7b-Instruct [41], CodeQwen1.5-7B-Chat [46], and Qwen2.5-Coder-7B [42]. (2) Models instruction-tuned for Verilog generation tasks (denoted as IT baselines), including ChipNeMo [24], Thakur et al. [22], RTLCoder [12], BetterV [13]

TABLE IV
COMPARISON UNDER SAME TRAINING SET SIZE.

Model	VerilogEval-Machine (%)			VerilogEval-Human (%)			RTLLM v1.1 (%)	
	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10	Syntax	Func.
RTLCoder-DeepSeek	61.2	76.5	81.8	**41.6**	50.1	53.4	**93.1**	**48.3**
RTLCoder-DeepSeek-Direct	59.8	73.6	77.2	39.1	48.3	51.3	86.2	44.8
CodeV-Verilog-DS-27K	**72.3**	**82.2**	**85.6**	39.1	**53.4**	**58.3**	89.6	44.8

and CraftRTL [15].

LLMs trained based on CodeLlama-7b-Instruct, DeepSeek-Coder-6.7b, CodeQwen1.5-7B-Chat, and Qwen2.5-Coder-7B will be denoted with the suffix CL, DS, CQ, QC, respectively.

Due to OriGen [17], AutoVCoder [14], and HAVEN [16] using techniques such as Self-Reflection, RAG, and CoT prompting models to enhance the performance of foundation models, which cannot be fairly compared with ours, so we have omitted the comparison with them.

B. Results on VerilogEval and RTLLM

In this section, we compare our method with previous work fairly on VerilogEval and RTLLM. Table III compares the main results of our CodeV with baseline methods on the VerilogEval and RTLLM benchmarks. We evaluate CodeLlama, DeepSeek-Coder, CodeQwen and Qwen2.5-Coder , while other baseline results are sourced from RTLCoder [12], BetterV [13], AutoVCoder [14], OriGen [17], CraftRTL [15] and HAVEN [16]. The results show that:

CodeV-Verilog performs well and achieves SOTA on Verilog-Machine. CodeV-Verilog performed well in the VerilogEval and RTLLM benchmarks, surpassing the RTLCoder and BetterV methods in the VerilogEval benchmark test, particularly in the most challenging pass@1 metric where CodeV-Verilog-QC achieves 80.1% in VerilogEval-machine, and CodeV-Verilog-QC achieves 59.2% in VerilogEval-human, significantly outperforming both GPT-4 and BetterV-CodeQwen. In the RTLLM v1.1 benchmark, CodeV-Verilog-CL nearly matches GPT-4's functional check success rate, failing in just 1 more case out of 29 circuits compared to GPT-4, and significantly outperforms all other models. Also, CodeV-Verilog significantly outperforms its GPT-3.5-turbo-0125 summarization model. These results demonstrate that contributed by a high-quality instruction tuning dataset, CodeV-Verilog exhibits significant superiority in Verilog generation tasks.

The performance of different base models becomes closer after fine-tuning. This is reasonable because the base models are trained on different training data but use the same fine-tuning data, which reduces their differences. Additionally, we find that after fine-tuning, CodeQwen surpasses DeepSeek-Coder, and CodeLlama also outperformed the other two models on VerilogEval-Machine pass@1 after fine-tuning. This indicates that the performance of a base model in specific domains does not necessarily align with its performance after fine-tuning.

Under the same dataset size, our method yields better data. As shown in Table IV, under the same data size (*i.e.* 27K), the results obtained from our approach surpass those of RTLCoder, especially for its directly trained version. Please note that in addition to data distillation, RTLCoder also proposes a training method based on syntax checking, which is orthogonal with our method and may improve CodeV-Verilog's performance in the future. More analytical results on the impact of data size are shown in Section V-E1.

TABLE V
PERFORMANCE OF MODELS ON CHISELEVAL AND CHISELTUTORIAL.

Model	ChiselTutorial (%)			ChiselEval (%)		
	p@1	p@5	p@10	p@1	p@5	p@10
GPT-3.5-turbo-0125	73.2	77.9	79.4	28.6	30.5	31.5
GPT-4	**79.1**	**89.5**	**91.2**	**38.9**	46.5	49.7
CodeLlama	31.2	56.7	63.2	17.7	26.2	29.4
DeepSeekCoder	33.8	53.0	58.0	3.0	5.6	6.9
CodeQwen1.5	69.4	80.3	82.2	25.3	34.3	36.8
Qwen2.5-Coder	53.5	79.3	83.9	19.3	34.8	40.2
RTLCoder-Mistral	0.3	1.4	2.9	0.0	0.0	0.0
RTLCoder-DS	30.5	37.7	41.1	1.6	3.3	4.1
CodeV-All-CL	76.2	80.0	81.7	34.3	40.6	42.7
CodeV-All-DS	66.8	81.3	85.0	32.6	41.2	43.5
CodeV-All-CQ	78.8	88.2	88.2	31.1	40.9	43.7
CodeV-All-QC	74.7	86.6	88.1	34.4	**47.3**	**51.0**

There is still a large gap between Human and Machine on VerilogEval. This stays the same as prior work. There are two reasons for this: First, the descriptions for Machine are generated by GPT, so they are closer to the distribution of LLMs compared with Human. Second, some of the Human descriptions contain tabular data like truth tables, which are difficult to obtain from the open-source codebase or GPT, leading to a distribution difference in the descriptions.

C. Results on Multi-lingual and Multi-Scenario Tasks

We evaluate LLMs' capabilities for multi-lingual and multi-scenario tasks in this section. As mentioned before, we added 18.7K Chisel data extension based on the existing 165K Verilog dataset, thereby constructing a more comprehensive training corpus, 184K-Verilog-Chisel. Then, we utilize the Chat-FIM-Tag supervised fine-tuning process to develop the CodeV-All series, which can handle tasks in different languages (Verilog and Chisel) and scenarios (Chat and FIM).

To evaluate the models' multi-lingual (Verilog and Chisel) generation capabilities on Chat tasks, we evaluate both our CodeV-All and baseline LLMs across four benchmarks: VerilogEval, RTLLM, ChiselTutorial, and ChiselEval. The results are presented in Tables III and V. To evaluate the models'

TABLE VI
COMPARISON BETWEEN OUR CODEV-ALL MODELS AND THE BASE MODELS ON CHISELEVAL-FIM.

Model	ChiselEval-FIM (%)								
	single-line			multi-line			random-span		
	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10
CodeLlama	72.0	82.6	85.0	36.4	42.7	44.4	33.3	37.9	39.5
DeepSeekCoder	68.9	76.7	78.7	35.0	40.0	41.2	40.6	43.9	44.6
CodeQwen1.5	79.8	86.7	88.1	47.2	51.5	52.3	42.6	48.3	50.0
Qwen2.5-Coder	83.4	88.3	89.4	47.6	52.7	54.1	45.9	50.3	52.0
CodeV-All-CL	88.3	90.4	90.9	45.8	48.1	48.9	48.2	51.8	52.7
CodeV-All-DS	89.6	**92.3**	**92.8**	**52.1**	**55.8**	**56.5**	**52.6**	**56.5**	**57.8**
CodeV-All-CQ	**89.7**	**92.3**	92.6	47.5	53.8	55.4	49.1	54.1	56.3
CodeV-All-QC	87.3	90.4	91.3	49.7	53.3	54.3	51.6	55.5	56.6

TABLE VII
COMPARISON BETWEEN OUR CODEV-ALL MODELS AND THE BASE MODELS ON VERILOGEVAL-FIM.

Model	VerilogEval-FIM-Machine (%)									VerilogEval-FIM-Human (%)								
	single-line			multi-line			random-span			single-line			multi-line			random-span		
	p@1	p@5	p@10	p@1	p@5	p@10	p@1	p@5	p@10	p@1	p@5	p@10	p@1	p@5	p@10	p@1	p@5	p@10
CodeLlama	57.6	65.2	67.3	40.0	48.5	51.2	42.2	50.4	52.5	56.7	64.5	66.0	39.9	47.7	50.0	33.9	40.1	41.6
DeepSeekCoder	65.8	69.5	70.7	45.6	48.7	49.7	51.6	55.0	55.7	63.1	68.1	70.3	43.8	49.0	50.6	43.2	48.7	50.0
CodeQwen1.5	59.6	68.4	71.8	41.9	48.1	50.7	41.9	48.1	50.2	55.1	64.3	66.6	34.8	41.9	43.2	38.6	43.9	45.8
Qwen2.5-Coder	71.2	74.2	74.6	48.1	54.2	55.4	53.3	58.4	59.9	67.2	70.7	71.4	44.8	48.5	49.3	42.1	47.8	49.7
CodeV-All-CL	76.9	78.9	79.6	63.6	67.0	67.8	61.1	66.4	68.0	75.6	78.1	78.6	55.4	58.7	60.0	52.2	56.3	57.6
CodeV-All-DS	74.3	80.0	82.2	63.2	69.5	71.2	65.2	69.2	71.0	71.2	75.6	77.2	59.2	65.0	66.8	58.9	63.9	65.5
CodeV-All-CQ	79.0	82.3	83.3	65.5	69.6	71.3	69.4	71.6	72.0	72.7	76.1	77.6	59.5	62.7	63.9	58.2	63.3	64.8
CodeV-All-QC	**80.0**	**84.3**	**85.5**	**72.8**	**74.8**	**75.5**	**70.5**	**73.4**	**73.9**	**76.4**	**79.1**	**80.0**	**62.3**	**66.5**	**67.6**	**64.7**	**68.4**	**69.3**

TABLE VIII
THE PERFORMANCE OF CODEV-ALL MODELS ON OUR CHISEL BENCHMARK.

Model	RTLLM-FIM (%)					
	single-line		multi-line		random-span	
	Syntax	Func.	Syntax	Func.	Syntax	Func.
CodeLlama	96.6	79.3	93.1	48.3	82.8	37.9
DeepSeekCoder	96.6	93.1	96.6	58.6	93.1	44.8
CodeQwen1.5	93.1	86.2	89.7	55.2	82.8	34.5
Qwen2.5-Coder	**100.0**	93.1	96.6	55.2	96.6	44.8
CodeV-All-CL	**100.0**	89.7	96.6	55.2	96.6	44.8
CodeV-All-DS	**100.0**	**96.6**	**100.0**	**72.4**	**100.0**	**62.1**
CodeV-All-CQ	**100.0**	93.1	**100.0**	58.6	**100.0**	55.2
CodeV-All-QC	**100.0**	**96.6**	**100.0**	62.1	93.1	58.6

TABLE IX
COMPARE THE PERFORMANCE OF THE MODELS TRAINED ON 165K VERILOG AND 184K VERILOG-CHISEL DATASETS

Model	ChiselTutorial (%)			ChiselEval (%)		
	p@1	p@5	p@10	p@1	p@5	p@10
Qwen2.5-Coder	53.5	79.3	83.9	19.2	34.8	40.2
CodeV-Verilog-QC(Ours)	0	0	0	0	0	0
CodeV-All-QC(Ours)	**74.7**	**86.6**	**88.1**	**34.4**	**47.3**	**51.0**

multi-scenario ability, we evaluate CodeV-All on the proposed FIM benchmarks. The results are shown in Tables VI, VII, and VIII. We can conclude that:

The inclusion of the Chisel dataset leads to an improvement in Chisel performance. To verify the efficiency of our method, we compare CodeV-Verilog and CodeV-All on ChiselTutorial and ChiselEval in Table IX. Compared to the data using only Verilog, our model significantly enhanced the Qwen2.5Coder's ability on Chisel, achieving a pass@1 of 74.7%, pass@5 of 86.6%, and pass@10 of 88.1% in the ChiselTutorial test, and a pass@1 of 34.4%, pass@5 of 47.3%, and pass@10 of 51.0% in the Chisel-Eval test.

TABLE X
ABLATION STUDY ON MULTI-LEVEL SUMMARIZATION (MLS) ON VERILOGEVAL-MACHINE AND VERILOGEVAL-HUMAN WITH PASS@K (P@K).

Model	Temp.	VE-Machine (%)			VE-Human (%)		
		p@1	p@5	p@10	p@1	p@5	p@10
CodeV-Verilog-CL w/o MLS	0.2	59.0	66.3	68.9	28.3	37.5	40.3
	0.5	59.5	72.1	75.8	25.1	40.4	45.7
	0.8	54.7	72.7	77.5	21.4	39.2	45.0
CodeV-Verilog-CL w/ MLS	0.2	**63.0**	71.2	72.9	**31.7**	38.1	39.8
	0.5	60.9	73.1	76.5	29.2	41.4	45.7
	0.8	58.5	**73.9**	**78.1**	25.7	**41.5**	**47.6**

CodeV-All demonstrates strong performance on Chisel generation tasks. CodeV-All-CQ achieves 78.8% on pass@1, 88.2% on pass@5, and 88.2% on pass@10 on the ChiselTurtrial benchmark. On the ChiselEval benchmark, CodeV-All-QC achieves 34.4% on pass@1, 47.3% on pass@5, and 51.0% on pass@10.

CodeV-All performs comparable to CodeV-Verilog on Verilog tasks. CodeV-All maintains high Verilog generation quality and grammatical correctness as shown in Table III. This finding suggests that the newly added Chisel data and FIM fine-tuning strategy don't negatively impact the model's

TABLE XI
THE ABLATION STUDY OF TAG ON THE 20K DATA.

Model	ChiselTutorial (%)			ChiselEval (%)			VerilogEval-Machine (%)			VerilogEval-Human (%)		
	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10
CodeV-All-QC-20K w/o Tag	70.0	86.4	88.1	29.2	39.6	44.1	67.2	75.5	78.2	42.9	54.2	59.1
CodeV-All-QC-20K w/ Tag	**78.8**	**91.0**	**93.4**	**30.5**	**40.8**	**44.6**	**69.9**	**77.0**	**80.2**	**46.3**	**56.8**	**60.7**

original Verilog generation capability, but rather effectively expand its functionality.

Our fine-tuning method is effective in improving the model's infilling performance. Due to the inconsistent and non-standardized infilling outputs generated by the base models in FIM tasks (all base models show poor performance), we apply post-processing to the outputs of the base models. That is, we identify and remove redundant modules, truncate the outputs at special tokens, and individually address certain unexpected outputs generated by the base models. It is important to note that CodeV-All does not engage in any post-processing beyond the removal of a redundant space character that may appear at the beginning of the output. The results show that CodeV-All achieves significant improvements across all three FIM benchmarks following the infilling fine-tuning process. This highlights the effectiveness of our method in enhancing the model's infilling capabilities, even when facing multi-lingual scenarios. Specifically, compared with the base models, CodeV-All achieves average performance improvements of 17.0% on VerilogEval-FIM-Machine and 13.7% on VerilogEval-FIM-Human (Table VII). On RTLLM-FIM, CodeV-All surpasses the base models by an average of 5.7% in syntax accuracy and 9.5% in functional correctness (Table VIII). Additionally, an improvement of 8.3% is observed on ChiselEval-FIM (Table VI).

D. Ablation Study

In this section, we will measure the effectiveness of multi-level summarization and adding language tags in fine-tuning. Due to budget constraints, we conduct this experiment only on a 10K Verilog dataset. We sample 10K Verilog data from the multi-level summarization dataset and generate single-level summarization (*i.e.* summarize directly) for the same Verilog code. The results in Table X show that the instruction data obtained through multi-level summarization can effectively enhance model performance which is consistent with our expectations.

To verify that adding tags helps LLMs in low-resource language, we train comparative LLMs on a mixed dataset of 20K Verilog and Chisel codes. The results, as shown in Table XI, demonstrate that adding tags can indeed improve LLMs' performance on both Chisel and Verilog.

E. Further Experimental Analysis

1) Impact of Data Scaling: We randomly sample subsets of various sizes (27K, 50K, 100K) from the complete 165K Verilog dataset and fine-tune all three baseline models, reporting their evaluation results on VerilogEval as shown in Figure 9. Except for CodeV-Verilog-CQ on VerilogEval-machine, which peaks on 100K-size, we observed that model performance generally improves as training set size increases, demonstrating the scalability of our methods.

TABLE XII
EVALUATIONS ON RTLLM USING PASS@K METRICS ($n = 20$).

Model	RTLLM v1.1 Syntax (%)			RTLLM v1.1 Func. (%)		
	p@1	p@5	p@10	p@1	p@5	p@10
CodeLlama	59.4	81.1	86.7	16.7	29.9	36.6
DeepSeek-Coder	75.6	89.0	92.2	29.8	40.1	44.8
CodeQwen	72.0	83.5	85.7	32.4	41.6	44.6
RTLCoder-Mistral	66.8	76.2	81.0	25.3	38.6	43.8
RTLCoder-DS	75.9	86.8	89.2	37.0	41.7	44.6
CodeV-Verilog-CL	81.7	92.3	93.0	40.8	52.0	54.9
CodeV-Verilog-DS	81.0	90.4	92.2	**43.9**	53.3	55.0
CodeV-Verilog-CQ	81.5	92.6	95.6	37.9	**55.1**	**63.4**
CodeV-Verilog-QC	**91.0**	**96.4**	**96.5**	42.4	51.8	57.4

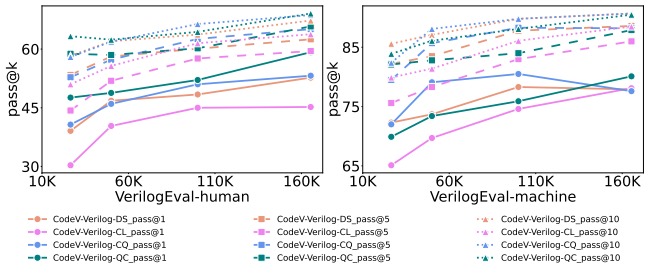

Fig. 9. Analysis on dataset sizes. We train CodeV-Verilog on different-size datasets and evaluate them on VerilogEval. The performance of models improves with the increase in training set size.

2) Impact of Sampling Temperature: Figure 10 further analyzes the VerilogEval results of our models when setting on different sampling temperatures. Generally, as the temperature increases, the pass@1 metric decreases while pass@5 and pass@10 increase. This is because when the temperature rises, the model tends to generate more uncertain and diverse responses, which has two effects: on the one hand, uncertain responses are more likely to produce incorrect results, lowering the accuracy of individual Verilog codes. On the other hand, the diversity enhances the exploration of various potential responses, thereby increasing the probability of at least one correct result being present among multiple Verilog codes.

3) A More Robust Evaluation on RTLLM with pass@k Metric: As shown in Eq. 3, the pass@k metric provides unbiased estimations. In this section, we set $n = 20$ to further evaluate our models on RTTLM using pass@k metrics and conduct varying comparisons with baselines. Table XII

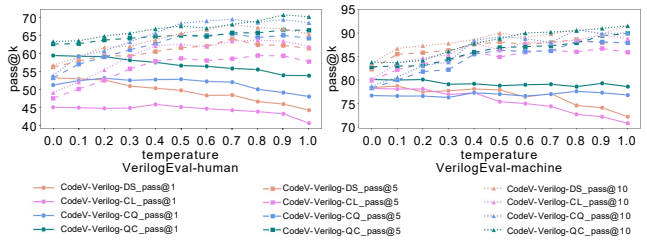

Fig. 10. Analysis on generation temperature. We test our models on VerilogEval, setting different temperatures. Generally, as the temperature rises, the pass@1 metric decreases while pass@5 and pass@10 increase.

illustrates that CodeV-Verilog performs better than all other baseline models, especially for functional correctness. Please note that there is a discrepancy in the pass@5 metric between Table XII and Table III. This difference is due to the varying sample sizes used in the two tables. We have thoroughly reviewed the results to ensure there are no errors.

VI. Conclusion

In this paper, we propose a fine-tuning pipeline, a series of open-source LLMs CodeV, a collection of open-source datasets, and several new benchmarks for multilingual (Verilog and Chisel) HDL generation in various scenarios (Chat and FIM). Our experimental results have shown CodeV's effectiveness with their superior performance over both previous open-source and commercial LLMs on Verilog and Chisel. This highlights its potential utility in the domain of processor design automation and we believe our work represents a significant step forward in the application of LLMs to HDL generation and will serve as a valuable resource for both academia and industry.

References

[1] J. Bergeron, *Writing Testbenches: Functional Verification of HDL Models*. Springer, 2003.

[2] H. Foster, "The 2022 wilson research group functional verification study," *Mentor Graphics*, 2022.

[3] J. Kabylkas, J. Smith, and R. Doe, "Effective processor verification with logic fuzzer enhanced co-simulation," in *Proceedings of the 2023 Design Automation Conference (DAC)*, 2023.

[4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.

[5] R. OpenAI, "Gpt-4 technical report. arxiv 2303.08774," *View in Article*, 2023.

[6] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," *arXiv preprint arXiv:2306.08568*, 2023.

[7] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Source code is all you need," *arXiv preprint arXiv:2312.02120*, 2023.

[8] Z. Yu, X. Zhang, N. Shang, Y. Huang, C. Xu, Y. Zhao, W. Hu, and Q. Yin, "Wavecoder: Widespread and versatile enhanced instruction tuning with refined data generation," *arXiv preprint arXiv:2312.14187*, 2023.

[9] N. Muennighoff, Q. Liu, A. Zebaze, Q. Zheng, B. Hui, T. Y. Zhuo, S. Singh, X. Tang, L. Von Werra, and S. Longpre, "Octopack: Instruction tuning code large language models," *arXiv preprint arXiv:2308.07124*, 2023.

[10] T. Zheng, G. Zhang, T. Shen, X. Liu, B. Y. Lin, J. Fu, W. Chen, and X. Yue, "Opencodeinterpreter: Integrating code generation with execution and refinement," *arXiv preprint arXiv:2402.14658*, 2024.

[11] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei *et al.*, "Starcoder 2 and the stack v2: The next generation," *arXiv preprint arXiv:2402.19173*, 2024.

[12] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution," *arXiv preprint arXiv:2312.08617*, 2023.

[13] Z. Pei, H.-L. Zhen, M. Yuan, Y. Huang, and B. Yu, "Betterv: Controlled verilog generation with discriminative guidance," *arXiv preprint arXiv:2402.03375*, 2024.

[14] M. Gao, J. Zhao, Z. Lin, W. Ding, X. Hou, Y. Feng, C. Li, and M. Guo, "Autovcoder: A systematic framework for automated verilog code generation using llms," 2024. [Online]. Available: https://arxiv.org/abs/2407.18333

[15] M. Liu, Y.-D. Tsai, W. Zhou, and H. Ren, "Craftrtl: High-quality synthetic data generation for verilog code models with correct-by-construction non-textual representations and targeted code repair," 2025. [Online]. Available: https://arxiv.org/abs/2409.12993

[16] Y. Yang, F. Teng, P. Liu, M. Qi, C. Lv, J. Li, X. Zhang, and Z. He, "Haven: Hallucination-mitigated llm for verilog code generation aligned with hdl engineers," *arXiv preprint arXiv:2501.04908*, 2025.

[17] F. Cui, C. Yin, K. Zhou, Y. Xiao, G. Sun, Q. Xu, Q. Guo, D. Song, D. Lin, X. Zhang *et al.*, "Origen: Enhancing rtl code generation with code-to-code augmentation and self-reflection," *arXiv preprint arXiv:2407.16237*, 2024.

[18] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[19] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[20] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Verilogeval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–8.

[21] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "Rtllm: An open-source benchmark for design rtl generation with large language model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 722–727.

[22] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking large language models for automated verilog rtl code generation," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2023, pp. 1–6.

[23] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.

[24] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu *et al.*, "Chipnemo: Domain-adapted llms for chip design," *arXiv preprint arXiv:2311.00176*, 2023.

[25] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[26] Y. Zhang, Z. Yu, Y. Fu, C. Wan *et al.*, "Mg-verilog: Multi-grained dataset towards enhanced llm-assisted verilog generation," *arXiv preprint arXiv:2407.01910*, 2024.

[27] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek, and K. Asanović, "Chisel: constructing hardware in a scala embedded language," in *Proceedings of the 49th Annual Design Automation Conference*, ser. DAC '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 1216–1225. [Online]. Available: https://doi.org/10.1145/2228360.2228584

[28] T. Liu, Q. Tian, J. Ye, L. Fu, S. Su, J. Li, G.-W. Wan, L. Zhang, S.-Z. Wong, X. Wang, and J. Yang, "Chatchisel: Enabling agile hardware design with large language models," in *2024 2nd International Symposium of Electronics Design Automation (ISEDA)*, 2024, pp. 710–716.

[29] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, sep 2023. [Online]. Available: https://doi.org/10.1109%2Fmlcad58807.2023.10299874

[30] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, "Chipgpt: How far are we from natural language hardware design," 2023.

[31] Y. Tsai, M. Liu, and H. Ren, "Rtlfixer: Automatically fixing rtl syntax errors with large language model," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[32] A. Nakkab, S. Q. Zhang, R. Karri, and S. Garg, "Rome was not built in a single step: Hierarchical prompting for llm-based chip design," in *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, ser. MLCAD '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3670474.3685964

[33] C.-T. Ho, H. Ren, and B. Khailany, "Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool," 2025. [Online]. Available: https://arxiv.org/abs/2408.08927

[34] M. ul Islam, H. Sami, P.-E. Gaillardon, and V. Tenace, "Aivril: Ai-driven rtl generation with verification in-the-loop," 2024. [Online]. Available: https://arxiv.org/abs/2409.11411

[35] S. Ranga, R. Mao, D. Bhattacharjee, E. Cambria, and A. Chattopadhyay, "Rtl agent: An agent-based approach for functionally correct hdl generation via llms," in *2024 IEEE 33rd Asian Test Symposium (ATS)*, 2024, pp. 1–6.

[36] E. Zhu, V. Markovtsev, A. Astafiev, A. Khan, C. Ha, W. Łukasiewicz, A. Foster, Sinusoidal36, S. Thakur, S. Ortolani, Titusz, V. Letal, Z. Bentley, fpug, hguhlich, long2ice, oisincar, R. Assa, S. Ibraimoski, R. Kumar, Q. TianHuan, M. J. Rosenthal, K. Joshi, K. Mann, JonR, J. Halliwell, and A. Oriekhov, "ekzhu/datasketch: v1.6.5," Jun. 2024. [Online]. Available: https://doi.org/10.5281/zenodo.11462182

[37] S. Williams, "Icarus verilog," https://github.com/steveicarus/iverilog, 2000.

[38] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[39] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013

[40] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen, "Efficient training of language models to fill in the middle," *arXiv preprint arXiv:2207.14255*, 2022.

[41] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming–the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

[42] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2. 5-coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[43] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "Ctrl: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[44] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, and M. Lewis, "Incoder: A generative model for code infilling and synthesis," *arXiv preprint arXiv:2204.05999*, 2022.

[45] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[46] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.

[47] N. Shazeer and M. Stern, "Adafactor: Adaptive learning rates with sublinear memory cost," in *International Conference on Machine Learning*. PMLR, 2018, pp. 4596–4604.

[48] K. Chang, K. Wang, N. Yang, Y. Wang, D. Jin, W. Zhu, Z. Chen, C. Li, H. Yan, Y. Zhou *et al.*, "Data is all you need: Finetuning llms for chip design via an automated design-data augmentation framework," *arXiv preprint arXiv:2403.11202*, 2024.

[49] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.